

Production Deployment Considerations

Corneel Moons, Eric Gutierrez, Samuel Fraley

June 28, 2026

1 Production Deployment Considerations

This project is designed as an end-to-end intelligent data driven application rather than only as a forecasting notebook. The solution combines forecasting, cloud deployment, data flow design, operational monitoring, and user-facing delivery. The objective is to generate weekly hourly demand forecasts for Glovo that can be executed in AWS and consumed by operations planners in a reliable and reproducible way.

1.1 Teams, Roles, Interactions, and Ways of Working

For a production-oriented intelligent application, the team should be organized as a small cross-functional group rather than as a purely analytical team. At a high level, three functional responsibilities are needed.

- **Analytics and modeling:** responsible for exploratory data analysis, feature engineering, baseline creation, model comparison, and leakage-safe time-series validation.
- **Platform and deployment:** responsible for cloud infrastructure, runtime packaging, storage design, permissions, scheduling, and execution of the forecasting job.
- **Delivery and coordination:** responsible for aligning outputs with business and assignment requirements, maintaining shared documentation, and preparing the final video and report.

These roles should interact through short feedback loops. Modeling decisions affect deployment feasibility, while deployment constraints may influence the choice of the production model. For that reason, the project follows an Agile style of work rather than a waterfall approach. Work is organized in small increments, reviewed iteratively, and tracked in a shared repository. In practical terms, the team worked through short weekly planning and review cycles, divided work into small deliverables, and used Git-based collaboration so that modeling, deployment, and reporting changes could be checked by another teammate before final integration. A common definition of done was used: code must run, outputs must be validated, and the resulting artifact must be documented clearly enough for another teammate to explain and reproduce it.

In practice, our team distributed these responsibilities across the three members. Corneel primarily covered analytics and the initial modeling work, Eric focused on the cloud deployment and S3-based execution path, and Sam supported delivery coordination and final submission readiness.

1.2 Rough Estimates and How They Were Agreed

The work was estimated at the level of major project components rather than individual coding tasks. This is more appropriate for a forecasting project, where uncertainty is high at the start

and becomes clearer after initial exploration. To keep the sizing consistent, the team used a simple relative three-level scale: low effort (roughly less than one focused work session), medium effort (around one to three work sessions), and high effort (multiple work sessions with iteration or uncertainty).

- **EDA and data quality review:** medium effort, because understanding timestamp gaps, zero-demand periods, and city-level consistency was necessary before modeling.
- **Baseline and candidate model development:** high effort, since several forecasting approaches had to be compared before selecting a deployable model.
- **Time-series validation and metric comparison:** high effort, because leakage-safe validation windows and assignment metrics had to be implemented and interpreted carefully.
- **Refactoring into a cloud-runnable workflow:** medium to high effort, as notebook-style code had to be simplified into a deterministic handler path.
- **AWS packaging, Lambda configuration, and S3 integration:** medium effort, including dependency layers, IAM permissions, storage paths, and runtime constraints.
- **Visualization and business-facing delivery:** low to medium effort, because the main user artifact is a forecast table or lightweight dashboard rather than a complex front-end system.
- **Video, report writing, and submission preparation:** medium effort.
- **Final testing and validation of outputs:** medium effort, including schema checks, 168-row validation, and end-to-end run verification.

These estimates were agreed collaboratively after reviewing the assignment requirements, the deployment examples, and the technical constraints of the runtime environment. The team first decomposed the project into major work packages, then discussed complexity, uncertainty, and dependency risk for each item until a shared relative size was reached by consensus. Instead of assigning exact hours too early, the team used relative sizing and updated assumptions as the project evolved. This reflects the Agile principle that estimates should be revisited as uncertainty decreases.

1.3 Architecture Components of the End-to-End Solution

The proposed architecture is intentionally simple and follows a serverless deployment pattern built around the actual production path implemented for this project. Rather than deploying a full application server, the forecasting workflow is packaged as a compact Lambda runtime artifact and executed on demand against files stored in Amazon S3.

- **Amazon S3** stores the input files, the latest forecast output, and timestamped forecast snapshots.
- **AWS Lambda** is the compute layer and runs the forecasting job from a zip deployment package.
- **Lambda layers** provide the heavy runtime dependencies such as `pandas` and `scikit-learn`, while the team-maintained zip only contains the project code and output checker.

- **IAM roles and policies** control read access to the S3 input path, write access to the forecast output path, and logging permissions.
- **Amazon CloudWatch** captures execution logs and supports operational monitoring of success and failure cases.
- **Amazon EventBridge** can trigger the job on a weekly schedule after the Sunday planning cut-off.
- **A lightweight visualization or reporting layer** can consume the latest forecast file from S3 and present it to planners.

In the implemented deployment strategy, the handler is `src.lambda_app.lambda_handler`. The deployment zip is built from the simplified `src/` runtime path plus `check_output_format.py`, and then uploaded to Lambda. At run time, the function downloads `train_data.csv` from S3 into temporary local storage, rebuilds the feature set, retrains the deployed serving model on all available history, generates the required 168-hour forecast, validates the output format, and uploads `predictions.csv` back to S3 both as the latest file and as a timestamped snapshot. This design maps the business workflow directly to cloud services and keeps the architecture transparent and reproducible.

1.4 Data Used and Data Flow

The main dataset is `train_data.csv`, which contains hourly timestamps, order counts, and city information for one market. During deployment testing, `test_data_mock.csv` can also be loaded to run the official output checker path and validate that the produced file conforms to the required schema.

The end-to-end data flow is as follows:

1. Historical demand data is stored in Amazon S3 under a configured input key.
2. The weekly forecast job is triggered either manually through a Lambda test event or automatically through a scheduler.
3. The Lambda handler reads configuration from the event payload or environment variables, then downloads the training file from S3.
4. The code preprocesses the data by sorting timestamps, reconstructing missing hourly periods, filling missing order counts with zero, and carrying forward the city field.
5. Feature engineering creates time-based, lag-based, rolling, holiday, weekend, trend, and zero-demand routing features.
6. The deployed serving model uses a hybrid strategy: a `HistGradientBoostingRegressor` handles the regular demand pattern, while a naive fallback is used for hours flagged by the zero-mask rule.
7. The production model generates forecasts for the next 168 hours, from Monday 00:00 to Sunday 23:00.
8. The output is validated to ensure correct row count, schema, datatype, time coverage, and CSV round-trip integrity. The optional checker path can also be executed when the mock test file is supplied.

9. The resulting `predictions.csv` is written back to S3 both as the latest forecast and as a timestamped snapshot for traceability and rollback support.
10. A visualization or reporting tool can then read the latest forecast file from S3 and present it to operations planners.

This flow reflects a DataOps-style design: data moves through controlled stages, with explicit validation before the result is used downstream. In end-to-end terms, the solution starts with historical demand data in cloud storage, transforms that data inside the forecasting service, writes the prediction artifact back to storage, and then exposes that artifact to a planner-facing reporting or visualization layer.

1.5 Environments to Be Used

The project distinguishes between development, testing, and production-style environments.

- **Development environment:** used for EDA, feature design, model experimentation, and metric comparison.
- **Testing environment:** used to validate the zip packaging step, Python 3.8 compatibility, Lambda handler configuration, dependency layers, S3 paths, and output format before the workflow is treated as production-ready.
- **Production environment:** the AWS execution path that reads data from S3, runs the weekly forecast inside Lambda, and stores the final predictions for downstream use.

This separation is important because experimentation and production execution have different goals. Development favors flexibility, while production favors reproducibility and reliability. In practical terms, the research notebooks can continue to evolve, while the deployed Lambda path remains intentionally narrower: one handler, one packaged runtime zip, fixed input and output paths, and controlled cloud permissions. Using distinct environments also aligns with standard DevOps and MLOps practice.

1.6 How the ML Model Will Be Built and Monitored

Model development begins in the research workflow, where the team compares a naive baseline with more advanced model families using leakage-safe walk-forward validation. Performance is evaluated with both MSE and SMAPE, since both metrics are required by the assignment and SMAPE is the main competition metric.

Once a model is selected for deployment, the cloud runtime uses a narrower serving path. In the current Lambda implementation, the deployed model is a hybrid forecaster built directly into the handler path. At execution time, Lambda retrains that serving model on all historical data available up to the forecast cut-off and then generates the next weekly forecast. Build reliability is supported through version-controlled code, a fixed zip-packaging process, handler-based validation checks, and a stable AWS configuration based on S3 keys, environment variables, and IAM permissions.

Monitoring happens at two levels:

- **Pipeline monitoring:** checks whether the Lambda ran successfully, whether the input file was available, whether the output file was written to S3, whether the returned row count was 168, and whether CloudWatch logs show any runtime errors.

- **Model monitoring:** checks whether forecast performance remains acceptable over time and whether prediction behavior changes in ways that suggest drift, degradation, or changing demand patterns.

If quality deteriorates materially, the team would rerun offline model comparison and consider promoting a new champion model into the Lambda serving path. This reflects the distinction between DevOps, DataOps, and MLOps: the software, the data pipeline, and the model all require separate but coordinated controls.

1.7 How End Users Will Use the Solution

The main end users are operations planners responsible for workforce and courier capacity planning. They do not need to interact directly with the model code or AWS configuration. Instead, they consume the forecast through an output artifact or a simple dashboard built on top of the forecast file.

The most basic user-facing version is a structured CSV or table containing forecasted hourly order volumes for the coming week. A more mature version would expose the same forecast through a dashboard, for example in Amazon QuickSight or another BI tool connected to the latest S3 output, highlighting daily totals, intraday peaks, and staffing-critical periods. In both cases, the forecasting solution acts as a decision-support system that helps planners move from manual estimation toward more systematic weekly planning.

1.8 Success KPIs

Success should be evaluated through both technical and operational KPIs. These KPIs should be measurable and linked to the intended production behavior of the system.

- **Forecast accuracy:** SMAPE on the held-out submission week, since it is the main assignment metric.
- **Supporting model metric:** MSE across validation windows and checked outputs, to ensure that model selection is not based on a single view of performance.
- **Operational reliability:** percentage of scheduled runs that complete successfully, ideally close to 100% for the weekly production job.
- **Data quality reliability:** percentage of runs where the input data passes preprocessing and validation without manual intervention.
- **Timeliness:** percentage of weeks where the forecast is available before operations planning begins.
- **Output validity:** percentage of runs that produce a valid 168-row CSV with the correct schema and time range.
- **Business usefulness:** reduction in manual forecast preparation effort, or positive planner feedback that the dashboard or forecast table is usable for staffing and capacity decisions.

Overall, the success of the system is not only defined by model accuracy, but also by whether it can reliably produce a valid, timely, and operationally useful forecast within the required cloud deployment framework.